



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 8
«Изучение эффективности модифицированных методов
быстрого умножения матриц»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать и исследовать эффективности модификаций методов Штрассена и Винограда-Штрассена быстрого умножения матриц.

2 Задачи

1. На основе уже реализованного в лабораторной работе №7 метода Штрассена необходимо выполнить его многопоточную версию.
2. Реализовать классическую версию метода Винограда-Штрассена.
3. Реализовать многопоточную версию метода Винограда-Штрассена.
4. Сравнить скорости умножения матриц всеми реализованными методами: в лабораторной работе №7 и лабораторной работе №8.
5. Построить на одной координатной плоскости разными цветами графики зависимости времени T (мс) умножения двух матриц размера $N \times N$ стандартным алгоритмом, алгоритмом Винограда, методом Штрассена, многопоточным методом Штрассена, методом Винограда-Штрассена, многопоточным методом Винограда-Штрассена от размера матрицы N . N изменяется от 2 до 1024. Выбор значения размера матрицы N необходимо осуществлять по формуле $N=2^n$, при этом изменяя n от 1 до 10 строго. Во время построения графиков на координатной плоскости на горизонтальной оси время, которое затрачено на выполнение алгоритма перемножения матриц строго в миллисекундах.

3 Практическая реализация

Код программы:

```
using LinearAlgebra
using Plots
using Base.Threads

function standard(A, B)
    n = size(A, 1)
    C = zeros(Float64, n, n)
    for i in 1:n
        for j in 1:n
            for k in 1:n
                C[i, j] += A[i, k] * B[k, j]
            end
        end
    end
    return C
end

function winograd(A, B)
    n = size(A, 1)
    C = zeros(Float64, n, n)
    m = n ÷ 2

    for i in 1:n
        for j in 1:n
            for k in 1:m
                C[i, j] += (A[i, 2*k-1] + B[2*k, j]) * (B[2*k-1, j] + A[i, 2*k])
                    - A[i, 2*k-1] * A[i, 2*k] - B[2*k, j] * B[2*k-1, j]
            end
        end
    end
    return C
end

"""
function strassen_false(A, B)
    n = size(A, 1)
    if n <= 2
        return standard(A, B)
    end
end
```

```

    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = strassen_false(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2

    A11 = A[1:k, 1:k]
    A12 = A[1:k, k+1:n]
    A21 = A[k+1:n, 1:k]
    A22 = A[k+1:n, k+1:n]

    B11 = B[1:k, 1:k]
    B12 = B[1:k, k+1:n]
    B21 = B[k+1:n, 1:k]
    B22 = B[k+1:n, k+1:n]

    P1 = strassen_false(A11 + A22, B11 + B22)
    P2 = strassen_false(A21 + A22, B11)
    P3 = strassen_false(A11, B12 - B22)
    P4 = strassen_false(A22, B21 - B11)
    P5 = strassen_false(A11 + A12, B22)
    P6 = strassen_false(A21 - A11, B11 + B12)
    P7 = strassen_false(A12 - A22, B21 + B22)

    C11 = P1 + P4 - P5 + P7
    C12 = P3 + P5
    C21 = P2 + P4
    C22 = P1 + P3 - P2 + P6

    C = zeros(Float64, n, n)
    C[1:k, 1:k] = C11
    C[1:k, k+1:n] = C12
    C[k+1:n, 1:k] = C21
    C[k+1:n, k+1:n] = C22

    return C
end
"""

function strassen(A, B)

```

```

n = size(A, 1)
if n <= 64
    return standard(A, B)
end
if n % 2 != 0
    new_size = n + 1
    A_new = zeros(Float64, new_size, new_size)
    B_new = zeros(Float64, new_size, new_size)
    A_new[1:n, 1:n] = A
    B_new[1:n, 1:n] = B
    C_new = strassen(A_new, B_new)
    return C_new[1:n, 1:n]
end

k = n ÷ 2

A11 = A[1:k, 1:k]
A12 = A[1:k, k+1:n]
A21 = A[k+1:n, 1:k]
A22 = A[k+1:n, k+1:n]

B11 = B[1:k, 1:k]
B12 = B[1:k, k+1:n]
B21 = B[k+1:n, 1:k]
B22 = B[k+1:n, k+1:n]

P1 = strassen(A11 + A22, B11 + B22)
P2 = strassen(A21 + A22, B11)
P3 = strassen(A11, B12 - B22)
P4 = strassen(A22, B21 - B11)
P5 = strassen(A11 + A12, B22)
P6 = strassen(A21 - A11, B11 + B12)
P7 = strassen(A12 - A22, B21 + B22)

C11 = P1 + P4 - P5 + P7
C12 = P3 + P5
C21 = P2 + P4
C22 = P1 + P3 - P2 + P6

C = zeros(Float64, n, n)
C[1:k, 1:k] = C11
C[1:k, k+1:n] = C12
C[k+1:n, 1:k] = C21
C[k+1:n, k+1:n] = C22

return C

```

```

end

function strassen_winograd(A, B)
    n = size(A, 1)
    if n <= 64
        return standard(A, B)
    end
    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = strassen_winograd(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2

    A11 = A[1:k, 1:k]
    A12 = A[1:k, k+1:n]
    A21 = A[k+1:n, 1:k]
    A22 = A[k+1:n, k+1:n]

    B11 = B[1:k, 1:k]
    B12 = B[1:k, k+1:n]
    B21 = B[k+1:n, 1:k]
    B22 = B[k+1:n, k+1:n]

    S1 = A21 + A22
    S2 = S1 - A11
    S3 = A11 - A21
    S4 = A12 - S2
    S5 = B12 - B11
    S6 = B22 - S5
    S7 = B22 - B12
    S8 = S6 - B21

    P1 = strassen_winograd(S2, S6)
    P2 = strassen_winograd(A11, B11)
    P3 = strassen_winograd(A12, B21)
    P4 = strassen_winograd(S3, S7)
    P5 = strassen_winograd(S1, S5)
    P6 = strassen_winograd(S4, B22)
    P7 = strassen_winograd(A22, S8)

```

```

T1 = P1 + P2
T2 = T1 + P4

C11 = P2 + P3
C12 = T1 + P5 + P6
C21 = T2 - P7
C22 = T2 + P5

C = zeros(Float64, n, n)
C[1:k, 1:k] = C11
C[1:k, k+1:n] = C12
C[k+1:n, 1:k] = C21
C[k+1:n, k+1:n] = C22

return C
end

function multi_treaded_strassen(A, B)
    n = size(A, 1)
    if n <= 64
        return standard(A, B)
    end
    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = multi_treaded_strassen(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2
    A11, A12 = A[1:k, 1:k], A[1:k, k+1:end]
    A21, A22 = A[k+1:end, 1:k], A[k+1:end, k+1:end]
    B11, B12 = B[1:k, 1:k], B[1:k, k+1:end]
    B21, B22 = B[k+1:end, 1:k], B[k+1:end, k+1:end]

    P1, P2, P3, P4, P5, P6, P7 = Tuple{Matrix{Float64}, Matrix{Float64},
        Matrix{Float64}, Matrix{Float64}, Matrix{Float64}, Matrix{Float64},
        Matrix{Float64}}(zeros(k, k) for _ in 1:7)

    Threads.@threads for i in 1:7
        if i == 1
            P1 .= multi_treaded_strassen(A11 + A22, B11 + B22)

```

```

elseif i == 2
    P2 .= multi_treaded_strassen(A21 + A22, B11)
elseif i == 3
    P3 .= multi_treaded_strassen(A11, B12 - B22)
elseif i == 4
    P4 .= multi_treaded_strassen(A22, B21 - B11)
elseif i == 5
    P5 .= multi_treaded_strassen(A11 + A12, B22)
elseif i == 6
    P6 .= multi_treaded_strassen(A21 - A11, B11 + B12)
elseif i == 7
    P7 .= multi_treaded_strassen(A12 - A22, B21 + B22)
end
end

C11 = P1 + P4 - P5 + P7
C12 = P3 + P5
C21 = P2 + P4
C22 = P1 - P2 + P3 + P6

C = [C11 C12; C21 C22]

return C[1:size(A, 1), 1:size(B, 2)]
end

function multi_treaded_strassen_winograd(A, B)
    n = size(A, 1)
    if n <= 64
        return standard(A, B)
    end
    if n % 2 != 0
        new_size = n + 1
        A_new = zeros(Float64, new_size, new_size)
        B_new = zeros(Float64, new_size, new_size)
        A_new[1:n, 1:n] = A
        B_new[1:n, 1:n] = B
        C_new = multi_treaded_strassen_winograd(A_new, B_new)
        return C_new[1:n, 1:n]
    end

    k = n ÷ 2

    A11 = A[1:k, 1:k]
    A12 = A[1:k, k+1:n]

```



```

A21 = A[k+1:n, 1:k]
A22 = A[k+1:n, k+1:n]

B11 = B[1:k, 1:k]
B12 = B[1:k, k+1:n]
B21 = B[k+1:n, 1:k]
B22 = B[k+1:n, k+1:n]

S1 = A21 + A22
S2 = S1 - A11
S3 = A11 - A21
S4 = A12 - S2
S5 = B12 - B11
S6 = B22 - S5
S7 = B22 - B12
S8 = S6 - B21

P1, P2, P3, P4, P5, P6, P7 = Tuple{Matrix{Float64}, Matrix{Float64},
    Matrix{Float64}, Matrix{Float64}, Matrix{Float64}, Matrix{Float64},
    Matrix{Float64}}(zeros(k, k) for _ in 1:7)

Threads.@threads for i in 1:7
    if i == 1
        P1 .= multi_treaded_strassen_winograd(S2, S6)
    elseif i == 2
        P2 .= multi_treaded_strassen_winograd(A11, B11)
    elseif i == 3
        P3 .= multi_treaded_strassen_winograd(A12, B21)
    elseif i == 4
        P4 .= multi_treaded_strassen_winograd(S3, S7)
    elseif i == 5
        P5 .= multi_treaded_strassen_winograd(S1, S5)
    elseif i == 6
        P6 .= multi_treaded_strassen_winograd(S4, B22)
    elseif i == 7
        P7 .= multi_treaded_strassen_winograd(A22, S8)
    end
end

T1 = P1 + P2
T2 = T1 + P4

C11 = P2 + P3
C12 = T1 + P5 + P6
C21 = T2 - P7
C22 = T2 + P5

```

```

        C = zeros(Float64, n, n)
        C[1:k, 1:k] = C11
        C[1:k, k+1:n] = C12
        C[k+1:n, 1:k] = C21
        C[k+1:n, k+1:n] = C22

        return C
    end

n = 100
A = randn(n, n)
B = randn(n, n)

standard_arr = standard(A, B)
winograd_arr = winograd(A, B)
strassen_arr = strassen(A, B)
strassen_winograd_arr = strassen_winograd(A, B)
multi_treaded_strassen_arr = multi_treaded_strassen(A, B)
multi_treaded_strassen_winograd_arr = multi_treaded_strassen_winograd(A, B)

error_winograd = maximum(abs.(standard_arr .- winograd_arr))
error_strassen = maximum(abs.(standard_arr .- strassen_arr))
error_strassen_winograd = maximum(abs.(standard_arr .- strassen_winograd_arr))
error_multi_treaded_strassen = maximum(abs.(standard_arr .-
    multi_treaded_strassen_arr))
error_multi_treaded_strassen_winograd = maximum(abs.(standard_arr .-
    multi_treaded_strassen_winograd_arr))

println("Погрешность алгоритмаВинограда ")
println(error_winograd)
println("Погрешность алгоритмаШтрассена ")
println(error_strassen)
println("Погрешность алгоритмаШтрассенаВинограда -")
println(error_strassen_winograd)
println("Погрешность алгоритмамногопоточногоШтрассена ")
println(error_multi_treaded_strassen)
println("Погрешность алгоритмамногопоточногоШтрассенаВинограда -")
println(error_multi_treaded_strassen_winograd)

N = [2^n for n in 1:10]
standard_times = Float64[]
winograd_times = Float64[]
strassen_times = Float64[]

```

```

strassen_winograd_times = Float64[]
multi_treaded_strassen_times = Float64[]
multi_treaded_strassen_winograd_times = Float64[]

for n in N
    A = randn(n, n)
    B = randn(n, n)
    print(n)
    println("x$n matrix counting...")

    standard_t = @elapsed standard(A, B)
    push!(standard_times, standard_t)

    winograd_t = @elapsed winograd(A, B)
    push!(winograd_times, winograd_t)

    strassen_t = @elapsed strassen(A, B)
    push!(strassen_times, strassen_t)

    strassen_winograd_t = @elapsed strassen_winograd(A, B)
    push!(strassen_winograd_times, strassen_winograd_t)

    multi_treaded_strassen_t = @elapsed multi_treaded_strassen(A, B)
    push!(multi_treaded_strassen_times, multi_treaded_strassen_t)

    multi_treaded_strassen_winograd_t = @elapsed
        multi_treaded_strassen_winograd(A, B)
    push!(multi_treaded_strassen_winograd_times,
        multi_treaded_strassen_winograd_t)
end

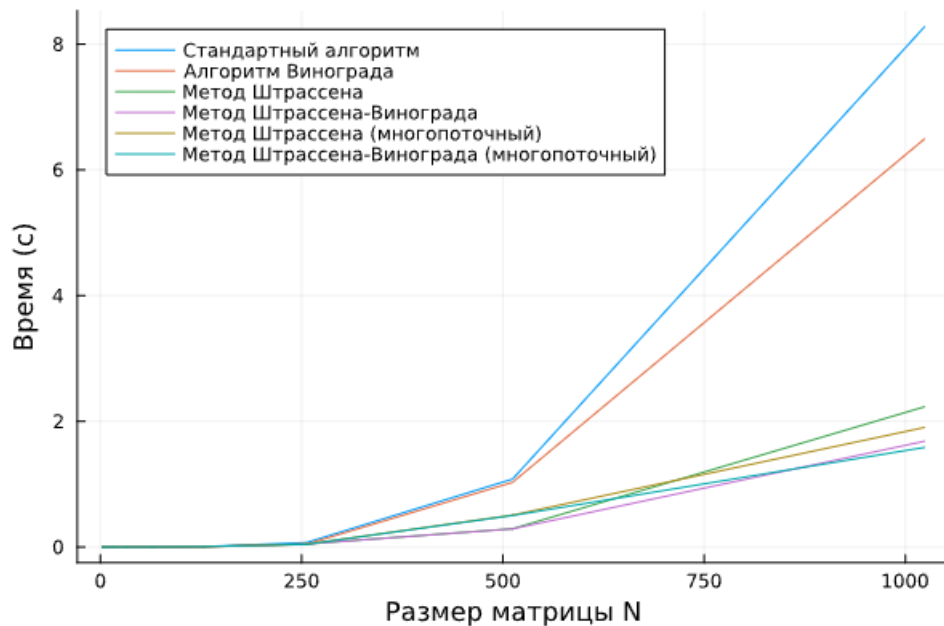
plot(N, standard_times, label="Стандартный алгоритм", xlabel="Размер матрицы N",
    , ylabel="Время c()", legend=:topleft)
plot!(N, winograd_times, label="Алгоритм Винограда")
plot!(N, strassen_times, label="Метод Штрассена")
plot!(N, strassen_winograd_times, label="Метод ШтрассенаВинограда-")
plot!(N, multi_treaded_strassen_times, label="Метод Штрассенамногопоточный ()")
plot!(N, multi_treaded_strassen_winograd_times, label="Метод
    ШтрассенаВинограда- многопоточный()")

```

4 Результаты

Результат выполнения программы:

Погрешность алгоритма Винограда
 $3.552713678800501e-14$
 Погрешность алгоритма Штрассена
 $4.618527782440651e-14$
 Погрешность алгоритма Штрассена-Винограда
 $4.973799150320701e-14$
 Погрешность алгоритма многопоточного Штрассена
 $4.618527782440651e-14$
 Погрешность алгоритма многопоточного Штрассена-Винограда
 $4.973799150320701e-14$



5 Выводы

В ходе выполнения лабораторной работы были реализованы и исследованы эффективности модификаций методов Штрассена и Винограда-Штрассена быстрого умножения матриц. Результаты выполнения показали, что в среднем самые лучшие результаты показывает метод Штрассена-Винограда многопоточный, затем метод Штрассена-Винограда и метод Штрассена многопоточный, однако на размере матриц 1024×1024 разница не столь существенна и результаты могут меняться. Возможно на более больших размерах результаты будут более конкретные.